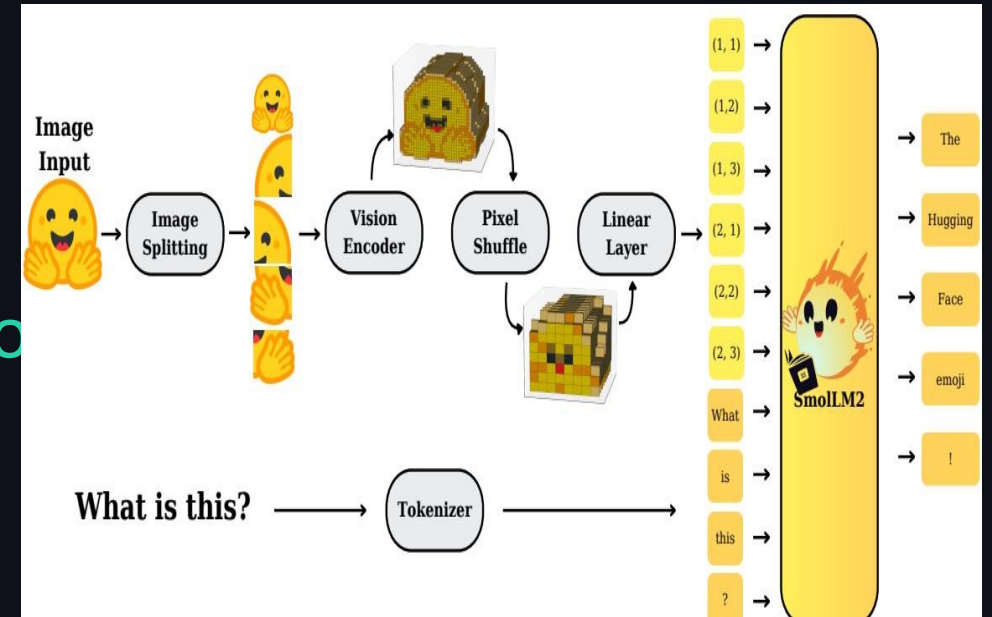


SmolVLM

Redefining Small and Efficient Multimodal

Hugging Face & Stanford University

Compact Vision-Language Models | Sub-1GB Inference | Fully Open Source



Large VLMs Are Powerful but Impractical for Edge Deployment

- ▶ State-of-the-art VLMs require 20–70 GB GPU memory (e.g., Molmo-A1B-7B: 27.7 GB)
- ▶ Massive parameter counts make real-time on-device inference impossible
- ▶ Mobile and IoT devices typically have < 8 GB shared memory
- ▶ Cloud-dependent inference introduces latency, privacy, and connectivity issues

Molmo-A1B-7B

27.7 GB RAM

7B params

InternVL2-2B

4.9 GB RAM

2B params

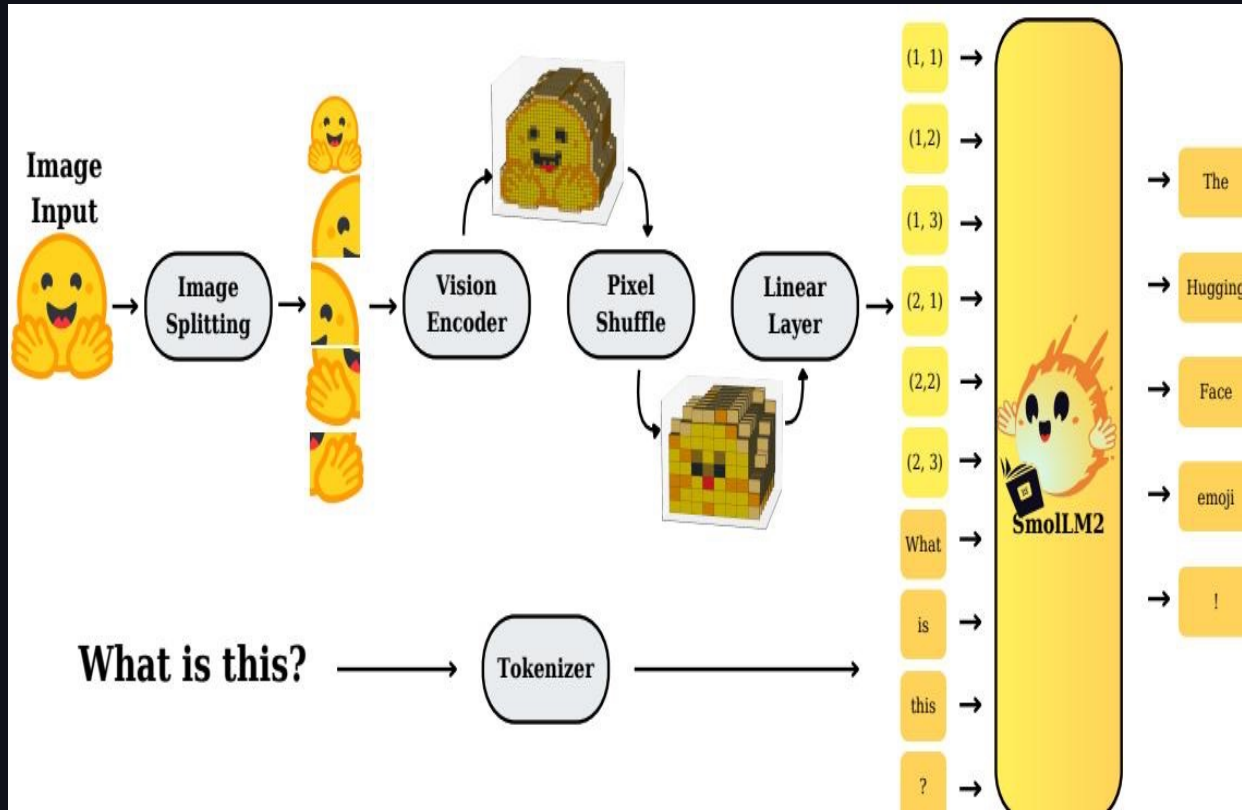
SmoVLM-256M

0.8 GB RAM

256M params

SmoVLM-256M uses 35× less memory than Molmo-A1B-7B while surpassing the 300× larger Llafacs-80B on key benchmarks

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmolLM2 Backbone



1

Image Splitting

Input image divided into sub-images for processing

2

SigLIP Vision Encoder

Compact encoder (93M or 428M) extracts visual features

3

Pixel Shuffle (r=2 or r=4)

Space-to-depth compression reduces token count by r^2

4

Linear Projection

Projects visual tokens into SmolLM2 embedding space

5

SmolLM2 Backbone

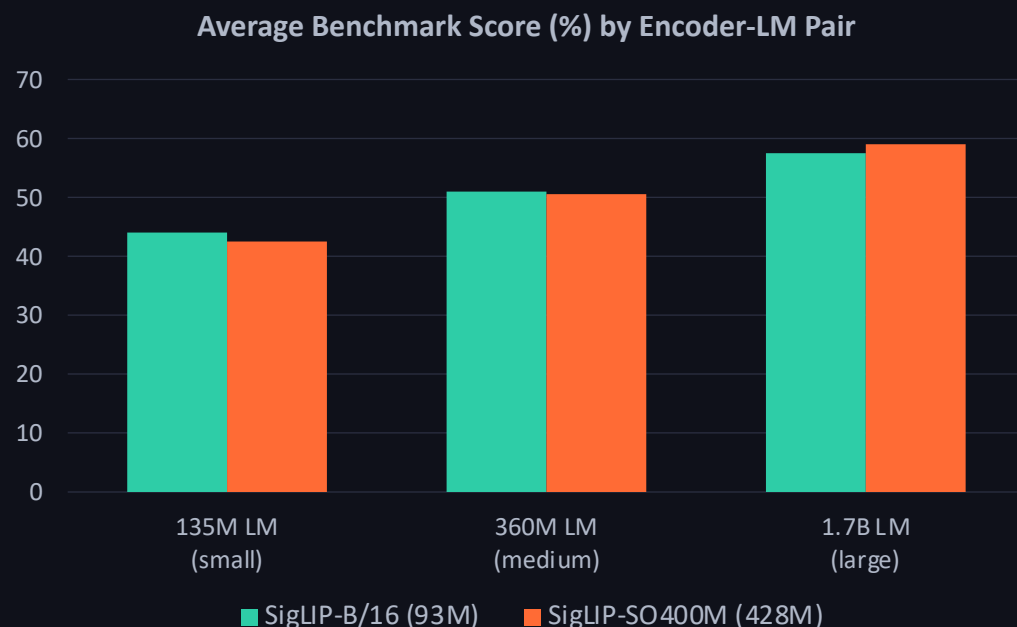
Three model sizes: SmolVLM-256M | SmolVLM-500M | SmolVLM-2.2B — All share the same architecture with scaled components. Language model processes concatenated visual + text tokens

Smaller Vision Encoders Outperform Larger Ones in Compact VLMs

Finding 1: Balanced Compute

A 93M SigLIP-B/16 paired with a 135M LM outperforms a 428M SigLIP-SO400M with the same LM.

Only at 1.7B LM scale does the larger encoder justify its 10% parameter overhead.



Small LM + Small Encoder

93M encoder wins: the LM capacity bottleneck means a larger encoder adds overhead without benefit

Large LM + Large Encoder

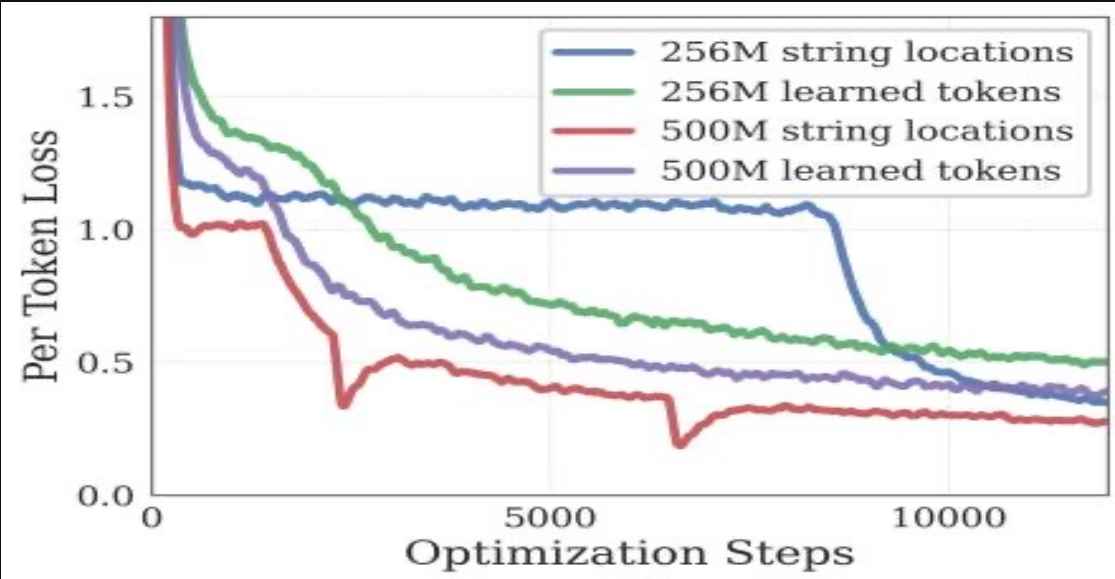
428M encoder only helps at 1.7B LM scale where the LM can exploit richer visual features

Design Principle

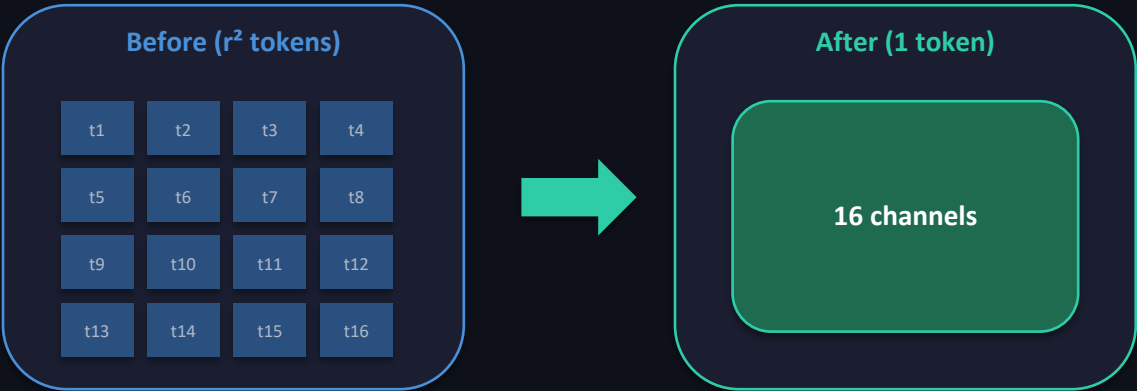
Match encoder capacity to LM capacity — oversized encoders waste compute in compact VLMs

Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models

Finding 3: Token Compression



Pixel Shuffle Mechanism



Compression Factor Comparison

Shuffle r	Token Reduction	Best For	Attention Savings
r = 2	4×	Large models (2.2B)	75% fewer tokens
r = 4	16×	Small models (256M/500M)	93.75% fewer tokens
No shuffle	1×	Baseline	None

Small VLMs benefit most from r=4: fewer tokens means lower attention overhead and better long-context modeling within tight memory budgets

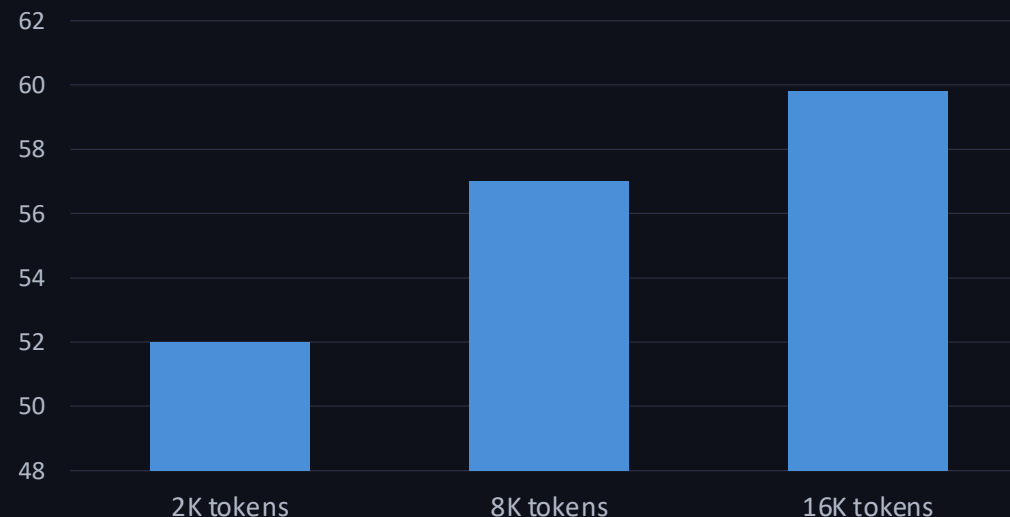
Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2: Context Scaling

Context Extension Strategy

- ▶ RoPE base increased from 10K to 273K for positional extrapolation
- ▶ Fine-tuned on mixed long/short-context data for stable adaptation
- ▶ 2.2B model uses 16K context; 256M and 500M use 8K
- ▶ Extended context enables higher-resolution image inputs without token overflow

Avg. Score vs. Context Length (2.2B)



SmolVLM-256M

8K context

Sufficient for standard image resolution

SmolVLM-500M

8K context

Balanced context for multi-frame video

SmolVLM-2.2B

16K context

Maximum resolution and multi-image support

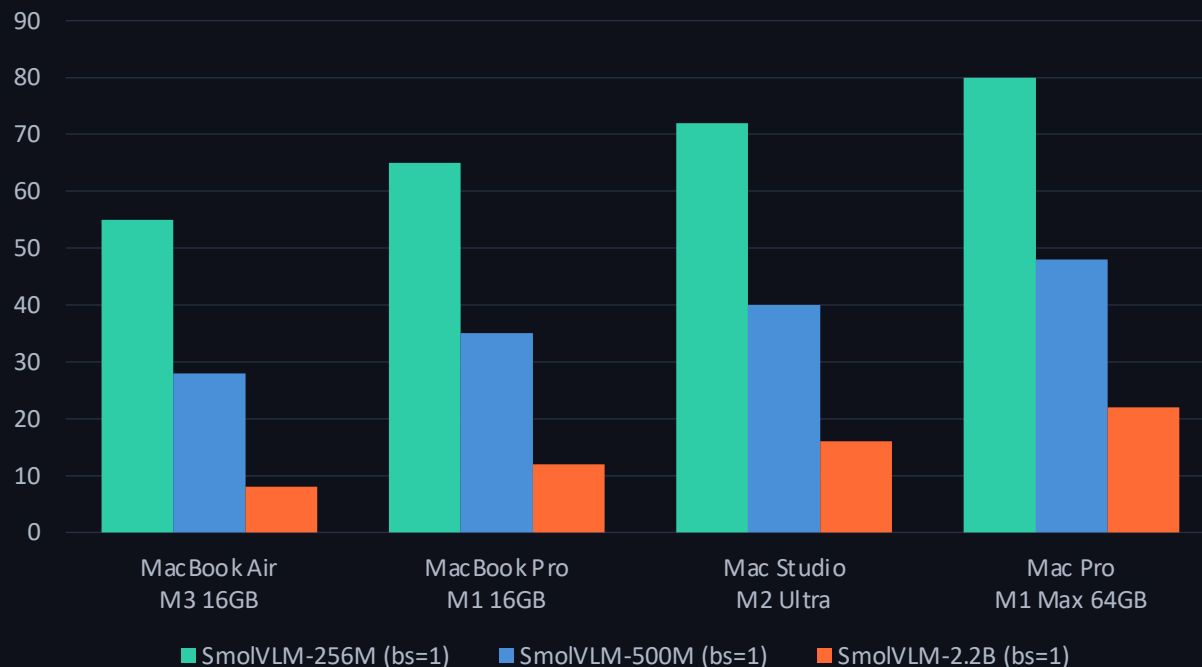
SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB RAM

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo-A1B-7B)
Single-Image	AI2D	46.4%	59.2%	70.0%	71.0% (Molmo-A1B-7B)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo-A1B-7B)
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo-A1B-7B)
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo-A1B-7B)
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo-A1B-7B)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo-A1B-7B)
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo-A1B-7B)
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo-A1B-7B)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2-2B)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2-2B)
	Average	44.0%	51.0%	59.8%	—

RAM: SmolVLM-256M = 0.8GB | SmolVLM-500M = 1.2GB | SmolVLM-2.2B = 4.9GB | Molmo-A1B-7B = 27.7GB

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon

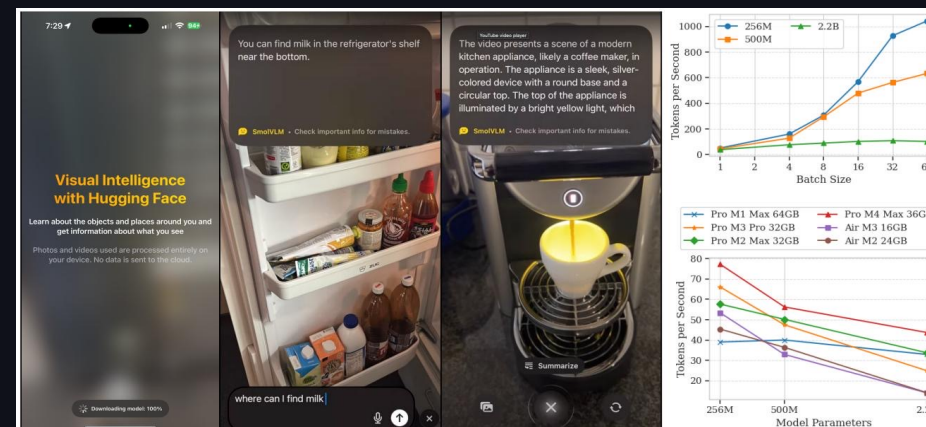
Tokens/Second by Device (Batch Size = 1)



Real-time multimodal inference on consumer hardware — enabling privacy-preserving AI applications without internet connectivity

HuggingSnap: On-Device iOS App

- Processes photos and video entirely locally — no cloud dependency
- SmolVLM-256M: up to ~80 tok/s on Pro M1 Max 64GB
- At batch=64: 256M model reaches ~1000 tok/s on high-end devices
- MacBook Air M3 16GB: 256M model produces ~55 tok/s
- Throughput scales roughly linearly with batch size for smallest model



Key Takeaways: Design Principles for Efficient Multimodal Models

1

Balance Encoder-LM Compute

Match vision encoder capacity to LM capacity. A 93M encoder outperforms a 428M encoder for small LMs — oversized encoders waste memory.

2

Extend Context Length Aggressively

Scaling from 2K to 16K tokens via RoPE base adjustment (10K→273K) and mixed-context fine-tuning unlocks higher-resolution inputs.

3

Use Aggressive Token Compression for Small Models

Pixel shuffle at $r=4$ delivers 16× token reduction, dramatically reducing attention overhead for compact VLMs under 500M parameters.

4

Sub-1GB Inference Is Achievable

SmolVLM-256M runs at 0.8 GB GPU RAM and surpasses the 300× larger LLaVA-72B — architectural efficiency beats raw scale.

5

Fully Open-Source Release

All model weights, training data, code, and the HuggingSnap mobile app are open-sourced to promote reproducibility and on-device AI research.

Efficient architecture × strategic design = competitive VLMs that run anywhere